Département SE / 3A (S5)
Programmation Structurée

Project: crossword generator

1 Preamble

The objective of this project is to implement a program that generates crosswords. We consider crosswords in a non-compact format, as depicted in Figure 1. The program picks words from a given lexicon and lays them out in a way that respects crossword rules. Note that, from a player point-of-view, the program actually generates the *solution* to a crossword. Furthermore, here we do not consider the problem of providing hints for the player to fill the crossword.

```
    crispy
       h
choreography
     r  u
     t  t
luckier h
     n  e
 films  n
        t
        i
        court
```

Figure 1: An example of generated crossword

2 Crossword generation

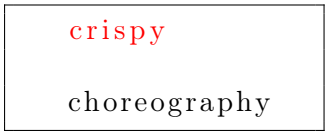
Generating a balanced crossword in an efficient manner is actually a difficult problem. We will now detail a simple brute-force solution. First, we assume that:

- The lexicon is read from a file;
- The program stores the crossword in a matrix (with many "empty" cells).

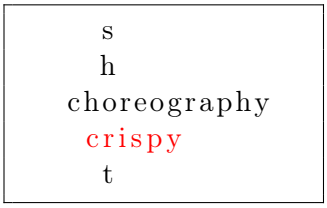
The crossword generation strategy works as follows:

1. Place one word from the lexicon in the matrix;
2. Pick a new word from the lexicon and iterate on each matrix cell until you find a *valid* starting cell for that word;
3. Repeat the previous step with new words.

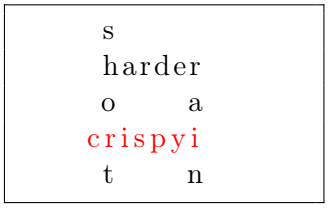
Figure 2 details a set of invalid crosswords, where the word depicted in red is laid out incorrectly. These examples should help you determine the criteria for a cell to be a valid starting cell for a given word. In these examples, we are trying to add the word "crispy" to an existing crossword.



(a) No intersection



(b) Conflict with neighbors



(c) Conflict at the end

Figure 2: Invalid crosswords

We provide some resources to ease your work in directory `~jforget/Cours/PS/Crossword`

- `lexicon.c`: a function to read a lexicon from a specified file;
- `lexicon-american`: an american lexicon. This lexicon is very large, and probably should not be used directly for your tests. Instead, you can generate a smaller random lexicon from it using command `shuf` (see `man shuf`).

3 Your work

You must write a command-line tool that generates crosswords. The tool must support the following features:

1. Print the crossword;
2. Generate a "dummy" crossword, that lays out words of the lexicon without checking for validity constraints. This is meant as a way for you to perform some simple preliminary tests. You must provide two different modes: in the *horizontal* mode, there is one word per line. In the *vertical* mode, there is one word per column;
3. Generate a crossword following the method described in the previous section. Check for validity constraints before adding a new word to the crossword;
4. *(Bonus)* Propose and implement improvements to the crossword generation method;
5. *(Bonus)* Add a *playable* mode. In this mode, when printing the crossword, letters are initially replaced by '*'. The words used for the crossword are also displayed (but not their position). The player wins when successfully guessing the start position of each word.

4 Evaluation

You must work as a pair with another student (binômes). No trio is allowed. This student must be in the same group as you (TP group). This implies that some students will work on their own. Evaluation will be more generous to them. We will evaluate your work based on a GitLab repository (see below for details).

4.1 Grading your work

Evaluation will consists of two parts:

1. Question/Answer session, December 19^{th}, 15 minutes per pair of student. We will discuss what you have done so far, based on the current content of your git repository. You must be able to explain every piece of code of your project.
2. At a later date, we will evaluate your code, according to the criteria detailed below. Evaluation will be based on the content of your git repository. The deadline for your last push will be decided later.

4.2 Your code

Concerning the git repository:

- Create a repository named `nom1_nom2_crosswords` (substitute with the names of the binôme members) on your GitLab account;
- Add the following members with role Developer to your repository:
  `iliana.fayolle@univ-lille.fr`
  `julien.forget@univ-lille.fr`
  `laurent.grisoni@univ-lille.fr`
  `luis.maldonado-saavedra.etu@univ-lille.fr`
  `florian.vanhems@univ-lille.fr`

The repository must contain:

- Your source code (**no compiled files**);
- Example lexicon files;
- A very short `README.md` (1 page max) that mentions:
  - How to use your program;
  - What is implemented and what is not implemented;
  - *(Optional)* The limitations of your program.

The following criteria will be taken into account to evaluate the quality of your code:

- Repository has frequent commits (at least once per session);
- Repository contains no unnecessary files;
- The program compiles without warnings with option `-Wall`;
- The program executes correctly, and causes no segmentation fault;
- Code is correctly indented;
- No magic numbers (check the web in case you do not know this term);
- Variable and procedure identifiers are explicit;
- Difficult points are briefly commented;
- Code is structured in procedures;
- No code duplication;
- Time/space complexity.